UpCharika Medical AI - medibot.py Code & Viva Guide

Comprehensive Line-by-Line Architecture Breakdown & Oral Defense Questions

1. Code Architecture: The 7 Core Subsystems of medibot.py

medibot.py contains 1,324 lines of Python code organized into 7 distinct functional modules. Here is the high-level roadmap of the entire file:

Module	Line Range	Core Responsibility	Key Functions / Classes
Part 1: Boot & CSS	Lines 1 - 44	UTF-8 encoding reconfigure, Streamlit widescreen layout, and custom CSS stylesheet injection.	sys.stdout.reconfigure() st.set_page_config() load_css()
Part 2: VectorStore & Drive	Lines 45 - 84	Google Drive download of vs.zip via gdown, memory caching with @st.cache_resource, and Cosine Similarity search.	load_vectorstore() get_vectorstore() retrieve_medical_context()
Part 3: Auth & Security	Lines 85 - 180	Encrypted cookies for persistent sessions, GROQ_API_KEY loader (.env & secrets), and bcrypt password hashing.	EncryptedCookieManager get_groq_client() register_user(), login_user()
Part 4: UI Card Styler	Lines 181 - 290	Cleans markdown for voice TTS and parses LLM text into color-coded clinical cards (Causes, Solutions, Drugs, Emergency).	clean_text_for_voice() format_medical_card()
Part 5: Offline Fallback KB	Lines 291 - 435	Built-in clinical guidance dictionary in English, Hindi, and Marathi when offline or Groq API is unavailable.	OFFLINE_CLINICAL_KB match_offline_kb()
Part 6: Groq RAG Engine	Lines 436 - 590	3-model fallback cascade (Qwen -> GPT-OSS -> Llama-3.1), textbook context prompt injection, and temperature=0.3.	call_groq_chat() get_ai_response() AVAILABLE_GROQ_MODELS
Part 7: UI & Features	Lines 591 - 1324	Google translation, gTTS voice audio, dynamic PDF medical report export, sidebar guides, and chat feed.	translate_text(), gTTS generate_medical_report() Chat Session State Loop

2. Part 1: Boot, Terminal Fixes & Styling (Lines 1?44)

```
# Lines 1-7: Reconfigure Windows terminal for UTF-8 (prevents emoji crashes)
import sys
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")

# Line 12: Streamlit Page Config MUST BE FIRST STREAMLIT CALL
st.set_page_config(page_title="UpCharika", layout="wide")

# Lines 16-22: Inject style.css for emerald glassmorphism
def load_css(file_name="style.css"):
    with open(css_path, "r", encoding="utf-8") as f:
        st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
```

Examiner Questions on Part 1:

Q: Why does st.set_page_config() have to be the very first Streamlit command?

Ans: Streamlit renders the HTML head tag and browser viewport first. Calling any other st command (like st.write or st.markdown) before set_page_config throws a StreamlitSetPageConfigFirstError.

Q: Why do you reconfigure sys.stdout to UTF-8?

Ans: Windows PowerShell defaults to cp1252 encoding. When printing status messages with Unicode characters or emojis, Python crashes with UnicodeEncodeError. reconfigure(encoding='utf-8') permanently prevents this.

3. Part 2: VectorStore & Google Drive Synchronization (Lines 45?84)

```
VECTOR_URL = "https://drive.google.com/uc?id=1UunmpU4iudJycj-OibngzcJf18nA6OmR"

def load_vectorstore():
    if not os.path.exists("vectorstore/db_faiss/index.faiss"):
        gdown.download(VECTOR_URL, "vs.zip", quiet=False)
        with zipfile.ZipFile("vs.zip", 'r') as zip_ref:
            zip_ref.extractall("vectorstore")
        os.remove("vs.zip")

@st.cache_resource(show_spinner=False)
def get_vectorstore():
    embed = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
    return FAISS.load_local("vectorstore/db_faiss", embed, allow_dangerous_deserialization=True)
```

Examiner Questions on Part 2:

Q: Why do you decorate get_vectorstore() with @st.cache_resource?
Ans: Streamlit re-executes the entire script from top to bottom on every user interaction. Without @st.cache_resource, loading the 198 MB FAISS index from disk on every keystroke would cause severe lag. @st.cache_resource keeps the loaded index in RAM permanently.

Q: Why is allow_dangerous_deserialization=True required in FAISS.load_local?
Ans: index.pkl uses Python's pickle protocol to serialize chunk metadata. LangChain requires this explicit flag to acknowledge that unpickling trusted local files is safe.

Q: Why do you retrieve k=2 chunks instead of 10?
Ans: k=2 extracts the top 2 highest-scoring chunks (~2,000 characters). This provides sufficient clinical context while keeping prompt token count low, minimizing latency on Groq LPUs.

4. Part 3: Security, Cookies & Patient Authentication (Lines 85?180)

```
USER_DB = "users.json"

def register_user(username, password):
    users = load_users()
    if username in users: return False, "Username already exists"
    # Encrypt password using salt + bcrypt hash
    users[username] = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
    save_users(users)
    return True, "Account created successfully"

def login_user(username, password):
    # Verifies bcrypt hash; has fallback for SHA-256 legacy users
    if bcrypt.checkpw(password.encode(), stored_val.encode()): return True, "Success"
```

Examiner Questions on Part 3:

Q: Why do you use bcrypt instead of plain SHA-256 or storing plain text?
Ans: bcrypt automatically incorporates a cryptographic salt (bcrypt.gensalt()) and uses a computationally expensive key derivation function. This makes dictionary attacks and rainbow-table cracking virtually impossible.

Q: Why do you use EncryptedCookieManager?
Ans: Streamlit's default session_state resets if the user refreshes their browser. Cookies allow persistent user authentication so patients remain logged in across page reloads.

5. Parts 4 & 5: Card Parser & Offline Fallback KB (Lines 181?435)

Lines 187?290: format_medical_card(text): Parses unstructured AI output into structured HTML badge cards. Lines starting with Causes get amber badges (#f59e0b), Care/Solutions get emerald badges (#10b981), Medications get cyan badges (#06b6d4), and Emergency warnings get rose borders (#f43f5e).

Lines 291?435: OFFLINE_CLINICAL_KB: High-precision medical triage rules for common clinical presentations (Fever, Headache, Cough, Acidity) translated natively into English, Hindi, and Marathi.

Examiner Question: What happens if the Groq API key is expired or internet goes down?
Ans: UpCharika has a dual offline fallback: first, it pulls verified excerpts directly from the local FAISS index (Textbook Mode); second, it queries the local rule-based OFFLINE_CLINICAL_KB dictionary. The patient is never stranded without clinical guidance.

6. Part 6: Groq LLM Multi-Model Engine & Clinical Prompt (Lines 436?590)

```
AVAILABLE_GROQ_MODELS = ["qwen/qwen3.8-27b", "openai/gpt-oss-20b", "llama-3.1-8b-instant"]

def call_groq_chat(client, messages, temperature=0.3):
    for m in AVAILABLE_GROQ_MODELS:
        try:
            return client.chat.completions.create(model=m, messages=messages, temperature=temperature)
        except Exception:
            continue  # Automatic failover cascade!

# Inside get_ai_response(): Strict Clinical Grounding
context_block = f"""Verified Medical Reference Context (from Clinical Textbook):
{context}
Carefully incorporate and ground your clinical advice in the above textbook findings."""
```

Examiner Questions on Part 6:

Q: Why is temperature set to 0.3 instead of 0.7 or 1.0?
Ans: Temperature controls randomness. High temperature (0.7-1.0) produces creative text suitable for stories but dangerous for medicine. Low temperature (0.3) enforces strict adherence to textbook facts, eliminating hallucinations.

Q: How does the multi-model cascade work?
Ans: If primary model Qwen hits a rate limit (HTTP 429), the try-except loop instantly passes the prompt to GPT-OSS or Llama-3.1 without dropping the user's connection.

7. Part 7: Voice, PDF Reports & Streamlit UI (Lines 591?1324)

Lines 594?650: Translation & Audio TTS: Uses deep_translator for Hindi/Marathi and gTTS to synthesize audio so illiterate or visually impaired patients can listen to remedies.

Lines 651?770: generate_medical_report(): Uses ReportLab to generate a downloadable clinical PDF summary containing patient demographics, chief complaints, symptoms, triage recommendations, and emergency hotlines.

Lines 771?1324: Chat Feed & Sidebar: Manages st.session_state.messages, quick symptom pills ('Fever', 'Migraine', 'Acidity'), emergency hotlines (108/112), and PDF guide download buttons.

Examiner Question: How does Streamlit remember the chat conversation history?
Ans: Streamlit uses st.session_state.messages (a list of dictionaries containing role and content). On every user query, the new message is appended to this list and re-rendered in the UI loop.